

If...Else Statements

It is used to execute one or more statement conditionally.

Syntax : 1) If <condition> then <statement>

Syntax : 2) If <condition> then
 <statement>

End if

Syntax : 3) If *condition* Then
 [statements]
 [ElseIf *condition-n* Then
 [elseifstatements] ...]
 [Else
 [elsestatements]]

End If

- If condition is true, VB .NET executes all the statements following the then keyword.
- You can use either single line or multiple-line syntax to execute just 1 statement.
- You can use comparison operators in the condition to generate a logical result that's true or false.
- If *condition* is **True**, the statements immediately following the **Then** keyword in the body of the **If** statement will be executed, and the **If** statement will terminate before the code in any **ElseIf** or **Else** statement is executed.

- If *condition* is **False**, the following **ElseIf** statements are evaluated, if there are any; this statement lets you test additional conditions, and if any are **True**, the corresponding code (*elseifstatements* above) is executed and the **If** statement terminates. If there are no **ElseIf** statements, or if none of their conditions are **True**, the code in the **Else** statement (*elsestatements* above), if there is one, is executed automatically.

Example:

```
Dim a, b, c As Integer
a = InputBox("Enter A: ")
b = InputBox("Enter B: ")
c = InputBox("Enter C: ")
If a>b And a>c Then
    MsgBox("A is Maximum")
ElseIf b>a And b>c Then
    MsgBox("B is Maximum")
Else
    MsgBox("C is Maximum")
End If
```

Select Case statement:

Executes one of several groups of statements depending on the value of an expression.

Syntax: Select Case *testexpression*
 [Case *expressionlist-n*
 [*statements-n*]]...
 [Case Else
 [*elasticsearch*]]
 End Select

- **Testexpression:** Required It may be any numeric or string expression.
- **Expression list_n:** it may be
 - expression or
 - expression To expression
 - Is expression operator expression
- **statements-n:** one or more statements executed if test expression doesn't match any of the case clause.
- If test expression matches any case expression list expression then statement following that clause is executed upto the next case clause or for the last clause or up to End select.
- After performing select...case, control executes the next statement after the End select.
- If test expression matches more than 1 expression then only the statements which follow the 1st match is executed.
- The Case Else clause is used to indicated the else statements to be executed if no match is found between test expression and expression list.

- Select case statement can be nested. Each nested select case statement must have a matching End select statement.

Example: Dim no1, no2 as integer

No2 =20

No1 = val (Inputbox (“Enter the value of no1:”))

Select case no1

Case 1:

Msgbox(“The value of no1 is:”) & no1

Case 2:

Msgbox(“The value of no1 is:”) & no1

Case 3 To 15:

Msgbox(“The value of no1 is between 3 to 5”)

Case Is > no2:

Msgbox no1 & “is grater than” & no2

Case else:

Msgbox (“nothing”);

End select.

With Statement

The **With** statement is not a loop, but it can be as useful as a loop. User can use the **With** statement to execute statements using a particular object.

Syntax: With *object*

[*statements*]

End With

Example:

With TextBox1

.Height = 1000

.Width = 3000

.Text = "Welcome to Visual Basic"

End With

Do Loop: The Do loop keeps executing its enclosed statements while or until *condition* is true. You can also terminate a Do loop at any time with an Exit Do statement. The Do loop has two versions; you can either evaluate a condition at the beginning:

Syntax: **Do [{While | Until} *condition*]**

 [*statements*]

 [Exit Do]

 [*statements*]

Loop

OR

Syntax: **Do**

 [*statements*]

 [Exit Do]

 [*statements*]

Loop [{While | Until} *condition*]

- A loop can be executed either while the condition is true or until the condition becomes true.
- When VB executes the loops, it first evaluates the condition. If the condition is false, the Do...While or Do...Until loop is skipped & execute the statement followed by the loop.
- The Do...Loop can execute any number of times as long as the condition is true.
- If the condition is initially false, the statements may never execute.
- Exit Do statement terminates the loop.

Example:

i=5	i=1
Do While i>0	Do until i<=5
Msgbox i	me.point i
i=i-1	i=i+1
loop	loop

For Loop

The **For** loop is probably the most popular of all Visual Basic loops. The Do loop doesn't need a *loop index*, but the **For** loop does; a loop index counts the number of loop iterations as the loop executes.

Syntax: For *index* = *start* To *end* [Step *step*]

 [*statements*]

 [Exit For]

[*statements*]

Next [*index*]

- The keywords in square brackets are optional.
- The keyword counter, start, end, increment all one of numeric type.
- **Index:** Required Numeric variable. The variable can't be a Boolean or an array element.
- **End:** Required find value of counter.
- **Step:** Optional Counter is changed each time through the loop. If not specified then step defaults to one.
- **Statement:** Optional one or more statement between for and Next that the executed the specified number of times.
- **Exit For:** to terminate the loop without executing the statements after exit for keyword.
- The loop is executed as many times as required for the counter to reach the end value.
- The increment argument can be either positive or negative. If start is greater than end, the value of increment must be negative. If not the loop's body can't be executed.

Example: for i=0 To 10

msgbox i

Next

for i=10 To 1 step -1

msgbox i

Next.

If I were to use a **step** size of 2:

For i = 0 To n Step 2

```
MsgBox(i)  
Next
```

For Each...Next Loop

You use the **For Each...Next** loop to loop over elements in an array or a Visual Basic collection. This loop automatically loops over all the elements in the array or collection.

Syntax: For Each *element* In *group*

```
    [statements]  
    [Exit For]  
    [statements]  
Next [element]
```

Example:

```
Dim intIDArray(3), intArrayItem As Integer  
intIDArray(0) = 0  
intIDArray(1) = 1  
intIDArray(2) = 2  
intIDArray(3) = 3  
For Each intArrayItem In intIDArray  
    System.Console.WriteLine(intArrayItem)  
Next intArrayItem
```


While Loop

While loops keep looping while the condition they test remains true, so you use a **While** loop if you have a condition that will become false when you want to stop looping.

Syntax: *While condition*
 [statements]
 End While

- If condition is true, all the statements are executed & when the wend statement is reached, control is returned to the While statement which evaluate condition again.
- If condition is still true, the process is repeated.
- If condition is false, the program executes the statement following Wend statement.

Example: i=5
 While i>=0
 total=total+i
 i=i-1
 End while

MsgBox Function

Syntax: MsgBox(*Prompt* [, *Buttons* As MsgBoxStyle =
MsgBoxStyle.OKOnly [, *Title*])

- **Prompt**—A string expression displayed as the message in the dialog box. The maximum length is about 1,024 characters (depending on the width of the characters used).
- **Buttons**—The sum of values specifying the number and type of buttons to display, the icon style to use, the identity of the default button, and the modality of the message box. If you omit *Buttons*, the default value is zero.
- **Title**—String expression displayed in the title bar of the dialog box. Note that if you omit *Title*, the application name is placed in the title bar.

You can find the possible constants to use for the *Buttons* argument in Table

<i>MsgBox</i> constants.		
Constant	Value	Description
OKOnly	0	Shows OK button only.
OKCancel	1	Shows OK and Cancel buttons.
AbortRetryIgnore	2	Shows Abort, Retry, and Ignore buttons.
YesNoCancel	3	Shows Yes, No, and Cancel buttons.
YesNo	4	Shows Yes and No buttons.
RetryCancel	5	Shows Retry and Cancel buttons.

MsgBox constants.

Constant	Value	Description
Critical	16	Shows Critical Message icon.
Question	32	Shows Warning Query icon.
Exclamation	48	Shows Warning Message icon.
Information	64	Shows Information Message icon.
DefaultButton1	0	First button is default.
DefaultButton2	256	Second button is default.
DefaultButton3	512	Third button is default.
ApplicationModal	0	Application modal, which means the user must respond to the message box before continuing work in the current application.
SystemModal	4096	System modal, which means all applications are unavailable until the user dismisses the message box.
MsgBoxSetForeground	65536	Specifies the message box window as the foreground window.
MsgBoxRight	524288	Text will be right-aligned.

MsgBox constants.

Constant	Value	Description
MsgBoxRtlReading	1048576	Specifies text should appear as right-to-left on RTL systems such as Hebrew and Arabic.

Example:

```
Private Sub Button1_Click
```

```
    Dim Result As Integer
```

```
    Result = MsgBox("This is a message box!", MsgBoxStyle.OKCancel  
+ MsgBoxStyle.Information + MsgBoxStyle.SystemModal, "Message  
Box")
```

```
    End If
```

```
End Sub
```

InputBox Function

User can use the **InputBox** function to get a string of text from the user.

Syntax: InputBox(Prompt [, Title [, DefaultResponse[,XPos [, YPos]]]])

- **Prompt**— A string expression displayed as the message in the dialog box. The maximum length is about 1,024 characters (depending on the width of the characters used).

- **Title**— String expression displayed in the title bar of the dialog box.
Note that if you omit **Title**, the application name is placed in the title bar.
- **DefaultResponse**— A string expression displayed in the text box as the default response if no other input is provided. Note that if you omit **DefaultResponse**, the displayed text box is empty.
- **XPos**— The distance in pixels of the left edge of the dialog box from the left edge of the screen. Note that if you omit **XPos**, the dialog box is centered horizontally.
- **YPos**— The distance in pixels of the upper edge of the dialog box from the top of the screen. Note that if you omit **YPos**, the dialog box is positioned vertically about one-third of the way down the screen.
- Input boxes let you display a prompt and read a line of text typed by the user, and the `InputBox` function returns the **string** result.

Example:

```
Private Sub Button3_Click
    Dim Result As String
    Result = InputBox("Enter your text!")
    TextBox1.Text = Result
End Sub
```

Procedure:

- A procedure is a set of one or more program statements that can be run or call by referring to the procedure name.
- We can divide the big program into small procedures.
- Due to procedure the application is easier to debug and easier to find error.
- The main advantage of procedure is its reusability.
- Procedures are categorized into two categories:
 - 1) In-built procedure
 - 2) User-defined procedure
- There are 4 types of procedure in VB.Net.
 - 1) Sub procedure: It does not return value.
 - 2) Event handling procedure: These are Sub procedures that execute in response to an event triggered by user action.
 - 3) Function procedures: they return a value.
 - 4) Property procedures: used in object oriented concept.

Sub Procedure: Also known as Sub Routine.

- Sub procedure may or may not have arguments. Arguments are not compulsory.
- A sub procedure does not return the value.
- By Val is by default argument type.
- To declare procedure Sub keyword is used.

Syntax: [**Private** | **Public**] **Sub** name [(parameterlist)]
 [statements]
 [**Exit Sub**]
 [statements]
End Sub

- **name** : It specifies name of the procedure.
- **parameterlist** : It is optional. It specifies the list of local variable names representing the parameters of this procedure. Specifies the parameters a procedure expects when it is called.
- Multiple parameters are separated by commas. The following is the syntax for one parameter.
`[optional] [ByVal | ByRef] varname[( )] [As type] [= defaultvalue]`
- **Optional**: Optional. Specifies that this parameter is not required when the procedure is called.
- **ByVal** : It is Optional. Specifies that the procedure cannot replace or reassign the variable element underlying the corresponding argument in the calling code.
- **ByRef** : It is Optional. Specifies that the procedure can modify the underlying variable element in the calling code the same way the calling code itself can.

- **parametername** : It is Required. Name of the local variable representing the parameter.
- **parametertype** : Required if **Option Strict** is **On**. Data type of the local variable representing the parameter.
- **defaultvalue** : Required for **Optional** parameters. Any constant or constant expression that evaluates to the data type of the parameter. If the type is **Object**, or a class, interface, array, or structure, the default value can only be **Nothing**.

Example:

```
Dim ans As Double
Private Sub sMsg()
    MsgBox("Hello")
End Sub
Private Sub sum1(ByRef n1 As Integer)
    MsgBox(n1 + 20)
End Sub
Private Sub butOk_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles butOk.Click
    Call sMsg()
    Call sum1(Val(Me.txtN1.Text))
End Sub
```


Function Statement:

Declares the name, parameters, and code that define a **Function** procedure.

Syntax:

[Public | Private] Function name [(arglist)] [**As** returntype]

[statements]

[name = expression]

[Exit Function]

[statements]

[name = expression]

End Function

- **name** : Required. Name of the procedure.
- **Parameterlist**: Optional. List of local variable names representing the parameters of this procedure.
- **returntype** : Required if **Option Strict** is **On**. Data type of the value returned by this procedure.
- Specifies the parameters a procedure expects when it is called. Multiple parameters are separated by commas. The following is the syntax for one parameter.

```
[ Optional ] [{ ByVal | ByRef }]    parametername[( )]  
[ As parametertype ] [ = defaultvalue ]
```

- **Optional** : Optional. Specifies that this parameter is not required when the procedure is called.
- **ByVal** : Optional. Specifies that the procedure cannot replace or reassign the variable element underlying the corresponding argument in the calling code.
- **ByRef** : Optional. Specifies that the procedure can modify the underlying variable element in the calling code the same way the calling code itself can.
- **parametername** : Required. Name of the local variable representing the parameter.
- **parametertype** : Required if **Option Strict** is **On**. Data type of the local variable representing the parameter.
- **defaultvalue** : Required for **Optional** parameters. Any constant or constant expression that evaluates to the data type of the parameter. If the type is **Object**, or a class, interface, array, or structure, the default value can only be **Nothing**.

Example:

```
Private Function res1(ByVal n1 As Double) As Double
```

```
    Return (n1 * 3.14)
```

```
End Function
```

```
Private Sub butOk_Click(ByVal sender As System.Object, ByVal e  
    As System.EventArgs) Handles butOk.Click
```

```
    ans = res1(Val(Me.txtN1.Text))
```

```
MsgBox("Result is:" & ans)  
  
End Sub
```

Continue : Transfers control immediately to the next iteration of a loop.

Syntax: Continue { Do / For / While }

User can transfer from inside a **Do**, **For**, or **While** loop to the next iteration of that loop. Control passes immediately to the loop condition test, which is equivalent to transferring to the **For** or **While** statement, or to the **Do** or **Loop** statement that contains the **Until** or **While** clause.

You can use **Continue** at any location in the loop that allows transfers.

If you have nested loops of different types, for example a **Do** loop within a **For** loop, you can skip to the next iteration of either loop by using either **Continue Do** or **Continue For**.

Example:

```
Dim c1 As Integer  
For c1 = 1 To 10  
    If (c1 Mod 2 = 0) Then Continue For  
    'If Counter is even then move to next item  
    MsgBox(c1)  
Next
```

Working with Windows Forms: In VB .Net forms are the base unit of the application. A form is ultimately a blank slate for a developer,

enhance with controls to create a user interface and with code to manipulate data.

Properties: following are common properties which are applied with almost each control.

1) Backcolor: Gets or sets the background color for the control.

Syntax: instance.BackColor = value

Instance: It contains name of the control with which this property is applied.

2) Cursor: Gets or sets the cursor that is displayed when the mouse pointer is over the control.

Syntax : instance.Cursor = value

Instance: It contains name of the control with which this property is applied.

3) Enabled: Gets or sets a value indicating whether the control can respond to user interaction.

Syntax: instance.Enabled = value

Instance: It contains name of the control with which this property is applied.

If it sets to true then the control can respond to user interaction. When it sets to false then user can't respond to the user interaction. The default is true.

4) ForeColor: Gets or sets the foreground color of the control.

Syntax: instance.ForeColor = value

Instance: It contains name of the control with which this property is applied.

5) FormBorderStyle: Gets or sets the border style of the form.

Syntax: instance.FormBorderStyle = value

Member name	Description
None	No border.
FixedSingle	A fixed, single-line border.
Fixed3D	A fixed, three-dimensional border.
FixedDialog	A thick, fixed dialog-style border.
Sizable	A resizable border.
FixedToolWindow	A tool window border that is not resizable. A tool window does not appear in the taskbar or in the window that appears when the user presses ALT+TAB. Although forms that specify FixedToolWindow typically are not shown in the taskbar, you must also ensure that the ShowInTaskbar property is set to false, since its default value is true.
SizableToolWindow	A resizable tool window border. A tool window does not appear in the taskbar or in the window that appears when the user presses ALT+TAB.

Instance: It contains name of the control with which this property is applied.

6) Icon: Gets or sets the icon for the form. An Icon that represents the icon for the form.

Syntax: instance.Icon = value

Instance: It contains name of the control with which this property is applied.

7) Locked: Locked property prevents a control from being moved at design time.

Syntax: instance.Locked = **value**

When it sets to True then user can't move control at design time. When it sets to False then user can move the control. By default it sets to be False.

Instance: It contains name of the control with which this property is applied.

8) Text: Gets or sets the text associated with this control.

Syntax: instance.Text = value

Instance: It contains name of the control with which this property is applied.

9) Visible: Gets or sets a value indicating whether the control and all its parent controls are displayed.

Syntax: instance.Visible = value

When it sets to true then the control and all its parent controls are displayed and when it sets to false then control and all its parent controls are not displayed. The default is true.

Instance: It contains name of the control with which this property is applied.

10) Windowstate: Gets or sets the form's window state. Specifies how a form window is displayed.

Syntax: instance.WindowState = value

Instance: It contains name of the control with which this property is applied.

The default is FormWindowState.Normal.

Member name	Description
Normal	A default sized window.
Minimized	A minimized window.
Maximized	A maximized window.

11) IsMdiContainer: Gets or sets a value indicating whether the form is a container for multiple-document interface (MDI) child forms.

Syntax: instance.IsMdiContainer = value

When it sets to true then the form is a container for MDI child forms and when it sets to false then the form is not a container for MDI child forms. The default is false

Instance: It contains name of the control with which this property is applied.

12) BackGroundImage: Gets or sets the background image displayed in the control.

Syntax: *instance.BackgroundImage = value*

Instance: It contains name of the control with which this property is applied.

Value contains an Image that represents the image to display in the background of the control.

13) StartPosition Property: Gets or sets the starting position of the form at run time.

Syntax: *instance.StartPosition = value*

Instance: It contains name of the control with which this property is applied.

Member name	Description
Manual	The position of the form is determined by the Location property.
CenterScreen	The form is centered on the current display, and has the dimensions specified in the form's size.

WindowsDefaultLocation	The form is positioned at the Windows default location and has the dimensions specified in the form's size.
WindowsDefaultBounds	The form is positioned at the Windows default location and has the bounds determined by Windows default.
CenterParent	The form is centered within the bounds of its parent form.

Methods: The following are common methods which are applied with almost all controls.

1) **Hide:** Conceals the control from the user.

Syntax: instance.Hide()

Instance: It contains name of the control with which this property is applied.

2) **Show:** Displays the control to the user.

Syntax: instance.Show()

Instance: It contains name of the control with which this property is applied.

3) **Focus:** Sets input focus to the control.

Syntax: instance.Focus()

Instance: It contains name of the control with which this property is applied.

4) **Close:** Closes the form.

Syntax: `instance.Close()`

Instance: It contains name of the control with which this property is applied.

Events: The events in the **lifecycle of a Form** from the time it is launched to the time it is closed are listed below:

- 1) **Load:** This event occurs before a form is displayed for the first time.
- 2) **Activated:** This event occurs when the form is activated in code or by the user. Occurs when the form is activated in code or by the user.
- 3) **Shown:** This event occurs whenever the form is first displayed
- 4) **Paint:** This event occurs when the control is redrawn.
- 5) **Resize:** Occurs when the control is resized.
- 6) **Deactivate:** This event occurs when the form loses focus and is not the active form. Occurs when an active form becomes inactive.
- 7) **Closing:** This event occurs when the form is closing.
- 8) **Closed:** This event occurs when the form is being closed.

Other Common Events:

- 1) **Click:** This event occurs when the control is clicked.
- 2) **GotFocus:** This event occurs when the control receives focus.
- 3) **LostFocus:** This event occurs when the control loses focus.
- 4) **KeyPress:** This event occurs when a key is pressed while the control has focus.

Working with SDI and MDI forms

Single Document Interface (SDI): SDI application allows only one open document frame windows at a time.

Multiple Document Interface (MDI):

- Only one MDI form is allowed in one project. MDI means Multiple Document Interface.
- MDI allows multiple document frame window to be opened in the same instance of an application.
- MDI form acts as the background for an application and can contain forms.
- MDI is most commonly used in those applications where the user can have many documents to work on at the same time.
- MDI is also handy when you have a large application.

Creating an MDI form :-

From Project Menu >> Add New Item >> Add MDI Parent form

OR

In Solution Explorer >> right click>> ADD >> MDI Parent Form

OR

Set IsMDIContainer property to true of the form.

Features of MDI form :

- All child forms are displayed within the MDI form's workspace. The user can move and size of child forms likes any other form, they are restricted to this workspace.

- When child is minimized, its icon appears on the MDI form instead of the taskbar. When the MDI form minimized, the MDI form and all of its child forms and all the child form are displayed in the same state they were before being minimized.
- When a MDI child form is maximized its caption is combined with the caption of the MDI form and is displayed in MDI's title.
- The active child form's menus are displayed on the MDI form's menu bar not on the child form.

Difference between - MDI & SDI :

	MDI	SDI
1	Multiple Document Interface	Single Document Interface
2	With the MDI, all of the IDE windows are contained within a single resizable parent window.	With the SDI, all of the IDE windows are free to be moved anywhere on the screen. As long as VB is the current application they will remain on top of any other application.
3	With MDI, windows can be docked to any side of the parent window.	With SDI, they can only be docked beneath the menubar.
4	One application contains only one MDI Form.	One application contains one or more SDI Forms.

5.	MDI is a Parent form.	SDI is Child form.
----	-----------------------	--------------------

Modules: These are also known as standard modules in Visual Basic. Modules are final and cannot be instantiated as object. Their members are shared and they become self-contained applications.

To Add Modules in Application:

From Solution Explorer → Right Click on Project → Add Module

The extension of Module is “.vb”

Ex: Module modConst

Public Const pi As Double = 3.14

End Module

Standard classes: The standard classes are used for all classes that form the building blocks of all .NET types and libraries. Standard classes are the basis of all reference types.

Namespace : It is a massive library of classes partitioned into many namespaces and packaged in a collection of assemblies.

- . Net framework class library is a collection of namespaces.
- Namespace is a logical naming scheme for types that have related functionality.
- Namespace means nothing but a logical container or partition.
- It's like Drives of the computer.

- For example: My computer contains C:, D:, E: and F: Each drive contains 1.txt file. The file 1.txt is available in all the drive so it is required to specify the drive name to locate the actual required file.
- At the top of the hierarchy is the System namespace.
- A namespace is just a grouping of related classes. It's a method of putting classes inside a container so that they can be clearly distinguished from other classes with the same name.
- A namespace is a logical grouping rather than a physical grouping. The physical grouping is accomplished by an assembly
- The .NET CLR consists of multiple namespaces, which are spread across many assemblies. For example, ADO.NET is the set of classes located in the System.Data namespace, and ASP.NET is the set of classes located in the System.Web namespace. In the CLR, the classes and structures contained in each of the namespaces represent a common theme of development responsibility.
- .NET Framework class library is collection of namespaces.
- Following table shows Common Namespaces supported by .NET

System	Contains fundamental classes and base classes.
System.IO	Contains classes for reading and writing data in file.
System.XML	Contains classes work with XML.
System.Windows.Forms	Contains classes for windows-based applications.
System.Data	Contains classes for the database connection.

Imports Statement: Imports namespaces or programming elements defined in the current project or in referenced projects and assemblies.

Syntax: Imports [aliasname =] namespace

OR

Imports[aliasname =] namespace.element

aliasname : It is optional. An *import alias* or name by which code can refer to *namespace* instead of the full qualification string.

namespace : It is required. The fully qualified name of the namespace being imported. Can be a string of namespaces nested to any level.

element : It is Optional. The name of a programming element declared in the namespace. Can be any container element.